(1) Algo. to Create linked list:

```
struct node {
        int data;
        struct node * next;
        };
        struct node * head, * new node, * temp;
        head = 0; int choice = 1;
    while (choice)
    {
new node = (struct node *) malloc (size of (struct node));
    printf (" enter data");
    scanf(" %d", & new node -> data);
    new node -> next = 0;
    if (head == 0)
    {
    temp=head = new node
    }
    else {
    temp -> next = newnode;
        temp = newnode;
    }
    printf ( do you want to Continue (0, 1)?)
        scanf ("%d", & choice);
}
```

(2) /* to reverse list */

```
temp = head;
while ( temp != 0)
    { printf ("%d", temp -> data);
        temp = temp -> next;
    }
}
```

(3) Find length of a linked list

```
lengthoflink ()
{
    int count;
    Struct node *temp;
    temp = head;

    while (temp != 0)
    {
        count = count +1;
        temp = temp -> next;
    }

    printf ("length is %d", count);

}
```

(4) Insert at beginning

→ Create a node first
→ Set pointers.

↓

```
newnode -> next <- head ;
           head <- newnode ;
```

(5) Insert at end

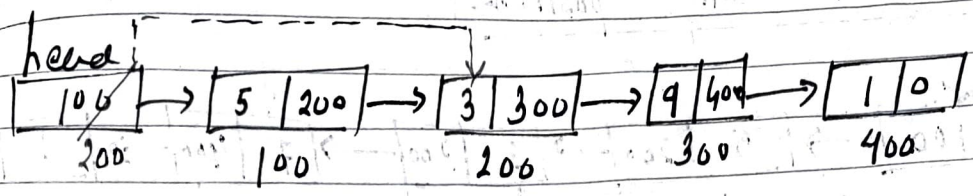
→ Create a node first.
→ Reach to the last node of list
→ Set pointers.

```
newnode -> next = 0;
temp = head;

while ( temp -> next != 0)
{
    temp = temp -> next;
}

temp -> next = newnode;
```

(6) Insert at given position

```
givenposition ( )
{
    int i, pos;
    struct node *temp;
if ( pos > count )   <- length of list
{
    printf ("invalid position");
}

Else
{
    temp = head;
while ( i < pos);
{
    temp = temp -> next;
    i = i+1;
}

newnode -> next = temp -> next;

temp -> next = new node;

}
```

Delete from linked list



(7) delete from beginning → update head pointer
→ free the memory

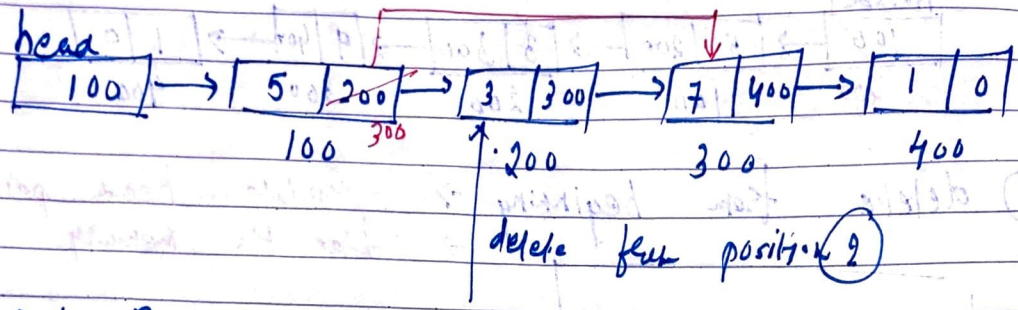
```
Delete from Beg. ( )
{
    struct node * temp;
    temp = head;
    head = head -> next;
    free (temp); }
}
```

```
if (head == 0)
{ pf ("list empty");
}
else
{
```

(8) Delete from end ( ) ← Delete from end

```
{ struct node * temp, * prevnode;
  temp ← head;
  while ( temp -> next != 0)
  {
      prevnode = temp;
      temp = temp -> next;
  }
if ( temp == head )
{
    head = 0;  free ( temp );
}
Else
{
    prevnode -> next = 0
    free ( temp )
```

(9) Delete from any position




```
Delete From pos ( )
  {
      Struct node * next node;
        int   pos, i;
        temp = head;
        printf (" enter position");
        scanf ("%d", & pos);
  while ( i < pos-1)
      {
            temp =   temp -> next
            i = i+1;
      }
      nextnode =   temp -> next;
      temp -> next =   nextnode -> next;
            free ( next node);
  }
```